



The **AI-native** modular workflow platform

12 changes across 3 phases — turning Blok into the only platform that is code-first *and* visual *and* multi-runtime *and* AI-native.

An AI assembles a production backend in a day — and a human opens the **same workflow** on a visual canvas.



Code-first

Typed, diffable, PR-reviewable
workflows — like Trigger.dev.



Visual

Connect-by-clicking on a canvas
— like n8n, but never fragile.



Multi-runtime

Nodes in 8 languages — like
Windmill, but the good parts stay
free.



AI-native

MCP + Skills over one CLI kernel.
Every rival bolted AI on after.

Every change, grouped by phase

PHASE 1 · FOUNDATION

The forks everything builds on

-  **S1 · Workflow JSON IR**
Promote Blok's v2 workflow shape to one published, versioned, canvas-aware JSON IR
-  **S2 · Versioned Node IDs**
Pin a node by scope and version right inside the step's use: ref
-  **S3 · Conditions Just Work**
Route branch and loop conditions through the same Mapper that step inputs already use.
-  **S7 · One Module Contract**
Extract one generic module-descriptor contract so triggers, runtimes, and nodes stop forking observability's code.

PHASE 2 · STUDIO · REGISTRY

Blok becomes editable & installable

-  **S4 · Editable Studio**
Studio gains a write path: edit workflows on the canvas, not just watch them run.
-  **S5 · Connect-Picker**
Click an upstream output field instead of hand-typing \$.state references that silently break
-  **S6 · The Blok Registry**
A Blok-owned registry that resolves, integrity-checks, and provenances every node before an AI installs it.
-  **S8 · Modular Triggers**
Add and remove triggers with one CLI command instead of hand-wiring framework internals.

PHASE 3 · MARKETPLACE · AI

The moat

-  **S9 · Multi-Runtime Nodes**
One node identity, N runtime implementations, installed and pinned by version instead of regex-patched.
-  **S10 · Managed Connections**
Nodes declare a connection; the runner injects the secret — never the workflow artifact
-  **S11 · AI-Native Surface**
Author errors enumerate the real step ids, so the AI self-corrects in one shot.
-  **S12 · Marketplace Trust**
Replace ungated `npm publish` with a server-side gate that auto-assigns a trust tier

01

Phase 1 — Foundation

The architecture forks. Mostly parallel, high-leverage. S3 ships first — it's bug fixes.



S1 Workflow JSON IR



S2 Versioned Node IDs



S3 Conditions Just Work



S7 One Module Contract

Promote Blok's v2 workflow shape to one published, versioned, canvas-aware JSON IR

Today

No version stamp, no canvas slot; VS Code validates it against a stale hand-written schema, Studio against nothing.

```
{
  "name": "Fetch and Respond",
  "version": "1.0.0",
  "trigger": { "http": { "method": "GET" } },
  "steps": [
    { "id": "fetch", "use": "@blokjs/api-call",
      "inputs": { "url": "https://example.com/api" } },
    { "id": "respond", "use": "@blokjs/respond",
      "inputs": { "body": "$.state.fetch" } }
  ]
}
```

Published IR (v2)

Same file plus a semver discriminant and a round-trippable ui slot, validated everywhere by one generated schema.

```
{
  "schemaVersion": "2",
  "name": "Fetch and Respond",
  "version": "1.0.0",
  "trigger": { "http": { "method": "GET" } },
  "steps": [
    { "id": "fetch", "use": "@blokjs/api-call",
      "inputs": { "url": "https://example.com/api" },
      "ui": { "x": 100, "y": 40 } },
    { "id": "respond", "use": "@blokjs/respond",
      "inputs": { "body": "$.state.fetch" }, "ui": { }
```

→ **Why.** One generated, addressable schema lets LLMs constrain-decode and self-validate, the canvas round-trips layout, and three-way schema drift becomes structurally impossible.

Pin a node by scope and version right inside the step's use: ref

• Today (schema v1)

No version, no scope guarantee — last month's green run can drift on upgrade.

```
workflow({
  name: "Fetch Countries",
  version: "1.0.0",
  steps: [
    // identity = one free-form string. No version.
    // install even strips the scope to derive the key
    { id: "fetch", use: "@blokjs/api-call" },
  ],
});
// runner: const h = nodes.getNode(node.node) // bare
// upgrade api-call -> behaviour silently changes
```

• Proposed (schema 2)

Mandatory scope kills typosquatting; the version pin travels in the ref itself.

```
workflow({
  name: "Fetch Countries",
  version: "1.0.0",
  schema: 2, // opt into scoped
  steps: [
    // @scope/name@range — one canonical, copy-paste-s
    { id: "fetch", use: "@blokjs/api-call@^1.2.0" },
  ],
});
// runner: nodes.resolve(node.node, nodeRange)
// semver.satisfies(version, "^1.2.0") or loud not-1
```

→ **Why.** Gives AI a trustworthy, scope-namespaced, version-pinned install identity — the foundation the registry (S6) and multi-runtime packaging (S9) build on.

Route branch and loop conditions through the same Mapper that step inputs already use.

● Today (v0.7)

The obvious condition silently 500s; only `eq()` is safe.

```
// A bare `$` in `when` compiles to "js/ctx.req.method"
// the if-else node evals it literally → ReferenceError
branch({
  id: "route",
  when: $.req.method === "POST", // silently throws
  then: [/ * ... */],
});

// The documented workaround – every author must remember
branch({ id: "route", when: eq($.req.method, "POST"),
```

● Proposed (S3)

One resolver for inputs, branch, loop — the obvious form is correct.

```
// Conditions resolve through the shared Mapper path (
// the pattern SubworkflowNode already ships) – the `j
// is stripped, `ctx.req` resolves against the live al
branch({
  id: "route",
  when: $.req.method === "POST", // just works
  then: [/ * ... */],
});

// loop({ while: $.state.hasMore }) works the same way
// eq() survives as convenience, not a required workar
```

→ **Why.** Kills a live silent-500 that every AI- and canvas-generated condition would reproduce; one mental model, plus a compiled-Function cache speeding large forEach/loop.

Extract one generic module-descriptor contract so triggers, runtimes, and nodes stop forking observability's code.

Today (welded to observability)

One union per surface; a cyclic dep freezes the whole CLI.

```
// closed union – every new surface forks this file
export type ObservabilityModuleId =
  | "tracing" | "metrics" | "logging" | "alerting" /*

// resolver INFINITE-LOOPS on a dependency cycle (A→B→A)
const out = new Set<ObservabilityModuleId>();
const visit = (id: string) => {
  const mod = getObservabilityModule(id);
  if (out.has(mod.id)) return; // catches diamc
  for (const dep of mod.dependencies) visit(dep); // i
  out.add(mod.id); // marked only
```

Proposed (S7)

One string-keyed contract; one Set turns the freeze into a clear error.

```
// one open contract – triggers/runtimes/nodes are jus
export interface ModuleDescriptor { kind: ModuleKind;

// generic resolver: back-edge throws instead of hang
const out = new Set<string>(), onPath = new Set<string>
const visit = (id: string) => {
  if (out.has(id)) return;
  const m = this.mods.get(id);
  if (onPath.has(id)) throw new Error(`Dependency cycl
  onPath.add(id);
  for (const d of m.dependencies) visit(d);
```

→ **Why.** S8 triggers and node-install become half-day data files, not subtree forks; the cycle hang and a transactional-apply bug get fixed once at the root.

02

Phase 2 — Studio · Registry · Triggers

The visible turn: Blok becomes editable and installable.

 S4 Editable Studio

 S5 Connect-Picker

 S6 The Blok Registry

 S8 Modular Triggers

Studio gains a write path: edit workflows on the canvas, not just watch them run.

● Today (v0.7)

You can see the graph but can't change a single step.

```
// Studio is a read-only telescope on runs
router.get("/workflows/:name", (req, res) => {
  const reg = WorkflowRegistry.getInstance().get(req.params.name);
  res.json({ definition: reg?.workflow }); // GET only
});

// Canvas renders the DAG but freezes it
<ReactFlow
  nodesDraggable={false}
  nodesConnectable={false}
/>
```

● Proposed (S4)

Drag, insert, rename, save — the canvas is the editor now.

```
// New write path — validate, then persist to source
router.put("/workflows/:name/definition", (req, res) => {
  const { definition, dryRun } = req.body;
  const { ok, errors } = validate(definition); // reuse validate
  if (!ok) return res.status(422).json({ errors });
  if (dryRun) return res.json({ ok, dag: buildWorkflow(definition) });
  persistToSource(reg.source, definition); // JSON write
  res.json({ ok: true, definition });
});
```

→ **Why.** Closes the see-edit-save loop in the browser; AI and humans edit the same JSON IR through one shared validator the runner already trusts.

Click an upstream output field instead of hand-typing \$.state references that silently break

● Today

Author guesses the upstream id and output path blind

```
{
  id: "respond",
  use: "@blokjs/respond",
  // hand-typed from memory: no autocomplete, no shape
  // a typo passes silently or 500s only on a full re-
  inputs: { body: $.state.fetch.data.id },
}
```

● Proposed

Picker emits a provably-valid reference, previewed before save

```
// Click the field in the connect-picker tree →
// Studio inserts a persistence-shape-correct ref,
// and POST /__blok/eval previews it live:
//   { ok: true, value: "user-123", type: "string" }
inputs: { body: $.state.fetch.data.id } // emitted, r
// schema + last-run trace ports; spread/as resolved i
```

→ **Why.** Kills the biggest silent-pass-through bug class: hand-typed references become one click, type-checked against schema and the live trace store before runtime.

A Blok-owned registry that resolves, integrity-checks, and provenances every node before an AI installs it.

● Today

Unpinned `npm install` — the AI's trust boundary is npm's reactive posture.

```
$ blokctl install stripe-charge

# package-manager.ts:70 — no version pin, floats to latest
INSTALL_NODE: (opts) => `npm install ${opts.node}`
#   → npm install @acme/stripe-charge (whatever "latest" is)
#   → npm runs the artifact's postinstall script
#   → regex-patches src/Nodes.ts to register it
# No scope guarantee, no immutability, no integrity hashes
# no provenance, npm-only (Go/Python nodes can't ship)
```

● Proposed

Scoped, pinned, hashed, provenanced — installable unattended by an AI.

```
$ blokctl install @acme/stripe-charge@^2.0.0

# Resolve against static Blok packument (CDN, no DB)
GET /@acme/stripe-charge          → versions, dist-tags
# Pick runtime-matched artifact, verify Blok-computed
#   integrity: "sha512-..."   provenance: Rekor #123456
# Publish gate already rejected: install scripts, versions
#   reuse, unowned scope. blok.lock pins (ref, ver, hash)
```



Why. An autonomous AI can install nodes safely: mandatory scopes kill typosquatting, zero-install-scripts neutralizes npm's #1 malware vector, and integrity plus provenance make installs reproducible.

Add and remove triggers with one CLI command instead of hand-wiring framework internals.

Today (v0.7)

Triggers are hardwired surgery; pure-HTTP installs drag in 4 unused packages.

```
// triggers/http/package.json – every HTTP
// project hard-depends on 4 unused siblings
"dependencies": {
  "@blokjs/trigger-mcp":      "^0.7.0",
  "@blokjs/trigger-sse":     "^0.7.0",
  "@blokjs/trigger-webhook": "^0.7.0",
  "@blokjs/trigger-websocket": "^0.7.0"
}
// To add a webhook: edit src/triggers/<kind>/index.ts
// patch package.json, set TRIGGER_<KIND>_PORT,
```

Proposed (S8)

One command installs, wires, and inventories a trigger — siblings become opt-in.

```
# Lean by default – http ships alone
blokctl create project --triggers http

# Opt in to a webhook receiver, one command:
blokctl trigger add webhook # installs pkg, deps,
                           # regenerates the mounts

blokctl trigger list --json # machine-readable inventory
blokctl trigger remove grpc # drops pkg + config + mounts
```

→ **Why.** Pure-HTTP projects stay lean, removal is real, and an AI agent can assemble trigger infra by command — mirroring the shipped `blokctl observability` pattern.

03

Phase 3 — Marketplace · AI · Distribution

The moat: multi-runtime distribution, the auth primitive, the AI-native surface.

 S9 Multi-Runtime Nodes

 S10 Managed Connections

 S11 AI-Native Surface

 S12 Marketplace Trust

One node identity, N runtime implementations, installed and pinned by version instead of regex-patched.

• Today (v0.7)

One npm tarball, hand-patched into Nodes.ts as a class instance — JS-only and unpinnable.

```
# JS-only, version-unaware, single runtime
$ blokctl install node @acme/validate-card

# under the hood: regex-patches src/Nodes.ts
const nodes: { ... } = {
  // appended by updateNodeFile():
  "validate-card": new ValidateCard() // class instance
};
# no version, no hash, no lockfile, no Go/Python
```

• Proposed (S9)

Same use: ref resolves to the runtime you enabled; blok.lock pins exact (version, hash).

```
# one identity, two real runtime impls, version-ranged
$ blokctl node install @acme/validate-card@^1.4 --runtime=go,python3

# writes blok.lock — the only new project file:
{ "nodes": {
  "@acme/validate-card": {
    "version": "1.4.0", "hash": "sha256-Yq3...",
    "runtimes": ["go", "python3"] } } }
# use: "@acme/validate-card@^1.4" resolves via lockfile
```

→ **Why.** An AI can resolve, install, and trust a node by (scope, name, version, hash) — and Blok's 8-runtime differentiator finally shows up in the product.

Nodes declare a connection; the runner injects the secret — never the workflow artifact

- **Today** Secret lives in ctx.env (default-open) — no reuse, no AI-wireable auth contract

```
defineNode({
  name: "stripe-charge",
  input: z.object({ amount: z.number(), currency: z.string() }),
  async execute(ctx, input) {
    const key = ctx.env.STRIPE_KEY; // open by default
    // ...
  },
});

// step: author hand-wires the secret channel, per node
{ id: "charge", use: "stripe-charge", ... }
```

- **Proposed** Artifact carries a name, not sk_live_...; secret stays off ctx.env and the trace

```
defineNode({
  name: "stripe-charge",
  connections: { stripe: { provider: "stripe" } },
  input: z.object({ amount: z.number(), currency: z.string() }),
  async execute(ctx, input) {
    const key = ctx.auth.stripe.apiKey; // injected,
    // ...
  },
});

// step binds by NAME — committable, shareable, AI-derivable
```

- **Why.** Unlocks the connector marketplace: connect once, reuse across 40 steps, rotate in one place, and let an MCP agent wire auth autonomously.

Author errors enumerate the real step ids, so the AI self-corrects in one shot.

- **Today (v0.7)** Generic hint never names the steps that DO exist — the AI loops.

```
// $.state.fetch references a step that doesn't exist.
// Mapper.guessHint (core/shared/src/Utils/Mapper.ts:1
// returns one generic string for every case:
return `the path `${parent}` is undefined or doesn't
  + `have a "${prop}" field at run time. Check the `
  + `trigger payload (ctx.req.body) or the upstream `
  + `step's output (ctx.state.<id>).`;
// → AI re-guesses, re-runs blokctl dev, reads the
// stack trace, guesses again. The footgun loop.
```

- **Proposed (S11 · M1)** Error names the exact available ids — the AI fixes it immediately.

```
// guessHint receives the normalized workflow's step id
// and lists them when the referenced id is unknown:
const known = stepIds.includes(id);
if (!known)
  return `ctx.state.${id} references no step. `
    + `Available step ids: [${stepIds.join(", ")}].`;
// → "ctx.state.fetch references no step. Available
// step ids: [validate, save]." Fixed in one shot,
// same message for the blok_test MCP loop + Studio
```

→ **Why.** Turns Blok's Zod schemas, JSON IR, and test harness into a self-correcting agent loop — the AI authors a backend in a day, not a debugging spiral.

Replace ungated `npm publish` with a server-side gate that auto-assigns a trust tier

- **Today (v0.7)** No floor, no provenance, no tier — an AI installs a typosquat blind

```
// publish/node.ts — anyone with a token publishes any
const registry = await registryManager.getRegistryToken
// long-lived _authToken written to a project .npmrc
fs.writeFileSync(npmrcFile,
  `registry=${URL}\n//${registry.url}:_authToken=${reg
// then: literal `npm publish`. No scan, no provenance
await exec(manager.PUBLISH({ registry: URL, npmrcDir:
// install side: no trust signal an AI can reason about
// blokctl install @acme/stripe-node → just runs it
```

- **Proposed (S12)** community floor + automated `verified` badge an agent can gate on

```
// server-side gate (S6 endpoint), reusing the validate
const floor = [resolvesToRealNode, noSecretsLeaked,
const verified = [noRuntimeDeps, provenancePresent, te

export async function evaluateTier(art): Promise<"comm
  for (const c of floor) await c.run(art); // thi
  try { for (const c of verified) await c.run(art); re
    catch { return "community"; } // bes
}
// resolve() now returns { tier, provenance } — one fl
// AI policy: auto-install verified, confirm community
```

→ **Why.** Lets an AI install on its own — safely. Mechanical tiers, OIDC tokens, zero install scripts; no human review queue, no relicensing time-bomb.

Smallest-shippable-first — each phase ends on a real demo

PHASE 1 · FOUNDATION

S1 IR · S2 identity · S3 expressions · S7 descriptor



Demo

A workflow validates against the published schema; the branch when bug fails at author-time, not as a silent 500.

PHASE 2 · STUDIO · REGISTRY

S4 edit · S5 UX · S6 registry · S8 triggers



Demo

Drag from an output → filtered palette → wire by clicking → run one step → publish a node. Zero hand-typed expressions.

PHASE 3 · MARKETPLACE · AI

S9 packaging · S10 connections · S11 MCP · S12 trust



Demo

An AI searches the registry, installs a Verified node by hash, wires a managed connection, runs tests green — all via MCP.

How Blok beats n8n, Trigger.dev & Windmill

CAPABILITY	N8N	TRIGGER.DEV	WINDMILL	BLOK
Code ↔ visual	Visual only, fragile	Code only	Visual + projections	Both, losslessly
Authoring ergonomics	Drag palette	Plain TS	Connect-picker	Both wins, one surface
Multi-runtime	Node only	Node only	Multi, gated triggers	Multi + free triggers
AI-native	None	NL over runs	Flow-builder chat	MCP + Skills kernel
Marketplace + auth	Two registries	"any npm pkg"	Hub + badge	npm-protocol + managed auth
Trust posture	Relicensed	—	Paywalled core	License set up front

Eight architecture calls — and three that are yours

CROSS-CUTTING DECISIONS (D1–D8)

- D1** TS canonical + published JSON IR
- D2** Canvas positions ephemeral; optional ui pass-through
- D3** npm-protocol-compatible registry
- D4** Mandatory scopes + version-pinned refs
- D5** Keep \$ proxy; fix the when footgun
- D6** One module descriptor for triggers/nodes/runtimes
- D7** blokctl is the single kernel
- D8** Multi-runtime = N impls, one entry

WHAT ONLY YOU CAN DECIDE

- 1 License & commercial model**
Open-core or permissive + paid trust tier — not the relicense move that cost n8n & Windmill their community.
- 2 Build vs. extend the registry**
Build the thin in-house registry — it gates the whole Phase-3 marketplace.
- 3 Accept the multi-runtime split**
"One node, all runtimes" → "one marketplace entry, N implementations." Turns multi-runtime into a sandboxing edge.

Ship **S3** now — it fixes a live bug, zero infra.
Prototype the **canvas**. Pick the **license**.

n8n is visual-but-fragile · Trigger.dev is code-but-not-visual · Windmill gates the good parts.
Blok is the only one that's all four — on a single kernel, with trust set before launch.